8

Dokumentowanie aplikacji

8.1. Komentarze

Komentarze to element każdego programu. Niezależnie od tego, w jakim języku programujemy, występują w nim komentarze. Ich głównym zadaniem jest opisywanie tworzonego kodu w celu ułatwienia późniejszego rozszyfrowania tego, co zostało napisane. Pisanie komentarzy jest szczególnie przydatne, gdy pracujemy w wieloosobowych zespołach i gdy planujemy automatyczne generowanie dokumentacji aplikacji.

Pisanie komentarzy powinno odbywać się w trakcie tworzenia kodu aplikacji. Spowoduje to, że będą one lepiej dopasowane do kodu i w każdej chwili będzie można wygenerować dokumentację.

Każdy język programowania ma swój własny sposób zapisywania komentarzy. W języku HTML komentarze są zapisywane pomiędzy znacznikami `<!--` i `-->`, w języku PHP, C++ i C# są zapisywane między znakami `/*` oraz `*/`.

Jeżeli planowane jest automatyczne generowanie dokumentacji tworzonej aplikacji, komentarze powinny być konstruowane w odpowiedni sposób. Powinny znaleźć się przy każdym dokumentowanym elemencie i zostać wyposażone w odpowiednie znaczniki. Takie komentarze należy zapisać w formacie DocBlocks. Jest to format najczęściej wykorzystywany przez generatory dokumentacji.

8.1.1. Komentarz w formacie DocBlocks

Komentarz tworzony w formacie DocBlocks jest zawsze ujęty w znaczniki `/**` i `*/`. Każdy wiersz między instrukcją otwierającą i zamykającą powinien zaczynać się gwiazdką `*`.

Aby wygenerować gotową dokumentację, dla każdego dokumentowanego elementu aplikacji należy stworzyć odpowiedni komentarz ujęty w znaczniki:

- Komentarze typu DocBlocks muszą się zaczynać od znaku `/**` i kończyć się znakiem `*/`.
- Każdy wiersz wewnątrz znaczników powinien zaczynać się gwiazdką `*`.
- Każdy komentarz poprzedza dokładnie jeden element konstrukcyjny, a cała zawartość komentarza ma zastosowanie do tego powiązanego elementu.

Komentarze powinny zawierać informację o tym, co robi dany fragment kodu, ale nie muszą opisywać dokładnie, co się dzieje w tym bloku programu. Powinny natomiast określać, co można za pomocą danego kodu osiągnąć.

Komentarz może na przykład opisywać argumenty metody oraz elementy zwracane. Dzięki temu użytkownik aplikacji łatwiej zrozumie przeznaczenie metody i wymagane dane.

Komentarze DocBlocks są podzielone na trzy części. Każda z nich jest opcjonalna, z tym że opis nie może istnieć bez podsumowania.

Podsumowanie

Podsumowanie, czyli krótki opis, jest wprowadzeniem do funkcjonalności powiązanego elementu. Podsumowanie kończy się, gdy:

- na końcu linii wystąpi kropka

lub

- wystąpiły dwa kolejne podziały linii.

Opis

Opis dostarcza więcej informacji dotyczących funkcjonalności powiązanego elementu. Mogą to być informacje na temat algorytmu funkcji, przykładu użycia funkcji lub tego, w jaki sposób klasa pasuje do całej architektury aplikacji. Opis kończy się po napotkaniu pierwszego znacznika lub po zamknięciu DocBlocks.

Tagi i adnotacje

Tagi i adnotacje dostarczają zwięzłych informacji (metadanych) o powiązanym elemencie. Mogą na przykład opisywać rodzaj danych zwracanych przez metodę lub funkcję. Każdy tag musi być poprzedzony znakiem `@` i musi zaczynać się od nowej linii. Oprócz nazwy tag może zawierać argumenty. Większość tagów jest powiązana z określonym typem elementu. Dlatego niektóre tagi dotyczą tylko klas, inne tylko metod. Najłatwiejszym sposobem weryfikacji, czy tag dotyczy określonego elementu, jest sprawdzenie dokumentacji jego opisu.

Przykład 8.1

Ogólna postać komentarza w formacie DocBlocks:

```
/**
 * Podsumowanie - informuje użytkownika o tym, co robi powiązany element.
 *
 * * Opis * - może rozciągać się na wiele wierszy, może dotyczyć
 * szczegółów powiązanego elementu i dostarczać pewnych dodatkowych
 * informacji.
 *
 * @param string argument - tagi i adnotacje mogą również
 * obejmować wiele wierszy.
 *
 * @return void
 */
  function mojaFunkcja(argument) {


  }
```

Tagi są zapisane za pomocą polecenia:

```
* @znacznik parametr1, parametr2, ...
```

Przykład 8.2

```
/**
 * @source
 * @param string argument To jest opis.
 */
```

Przykład 8.3

Najprostszy komentarz może zawierać wyłącznie opis słowny całego elementu:

```
/** Ta metoda wysyła dane na ekran. */
public void Pisz() {
  Console.WriteLine("Nazwisko:  " + nazwisko);
}
```

Przykład 8.4

Komentarz może opisywać argumenty metody oraz elementy zwracane:

```
/**
* Metoda sprawdza, czy ktoś wygrał grę.
*
* Zwraca true, jeśli gra się zakończyła.
* @param wynik zawiera stan gry.
* @return boolean zwraca stan gry.
*/
public void KtoryGracz(int wynik) {
  if (wynik == 3 || wynik == 30)
  {
    koniec = true;
  }
}
```

W podanym przykładzie zostały użyte dwa znaczniki — `@param` i `@return`. W znaczniku `@param` została podana dodatkowo nazwa argumentu, którego znacznik dotyczy. Znacznik `@return` zawiera informację o typie zwróconej wartości.

8.2. Tworzenie dokumentacji programu

8.2.1. Wprowadzenie

Tworzenie dokumentacji aplikacji jest bardzo ważną częścią procesu programowania. Dobrze napisana dokumentacja oprogramowania powinna zawierać również informacje o sposobie korzystania z aplikacji. Może też służyć jako źródło informacji o przebiegu procesu tworzenia oprogramowania.

8.2.2. Tworzenie dokumentacji

Prawidłowo opracowana dokumentacja powinna obejmować następujące dokumenty:

- dokumentację wymagań dla systemu,
- dokumentację projektu oprogramowania,
- dokumentację techniczną,
- dokumentację użytkownika.

Uzupełnieniem całościowej dokumentacji może być dokumentacja przeznaczona dla administratorów systemów.

Dokumentacja wymagań dla systemu

Dokumentacja wymagań dla systemu jest istotnym narzędziem dla projektantów oprogramowania, programistów i testerów. Powinna zawierać wszystkie funkcjonalne i niefunkcjonalne wymagania dla tworzonego oprogramowania. Dokumentacja ta jest podstawą do opracowania oprogramowania oraz do testowania i weryfikacji tworzonej aplikacji. Może zawierać dokumenty dotyczące projektu oprogramowania, raporty z przeprowadzonych rozmów z użytkownikami aplikacji, kwestionariusze i wyniki badań.

Dokumentacja wymagań często jest przygotowywana przez klienta zlecającego tworzenie aplikacji.

Dokumentacja projektu oprogramowania

Dokumentacja projektu oprogramowania powinna zawierać wszystkie informacje potrzebne do zbudowania oprogramowania. W jej skład powinny wchodzić:

- opis architektury oprogramowania,
- szczegółowy projekt oprogramowania,
- diagramy przepływu danych,
- projekt bazy danych.

Zgromadzone dokumenty są informacją dla programistów, jak korzystając z wytworzonego oprogramowania, można je rozwijać i modyfikować. Powstałe dokumenty nie zawierają szczegółów dotyczących kodowania programu, ale powinny zawierać informacje niezbędne do tworzenia oprogramowania.

Dokumentacja projektu najczęściej jest przygotowywana przez analityków biznesowych.

Dokumentacja techniczna

Dokumentacja techniczna jest przeznaczona dla osób, które będą administrowały aplikacją. Zawiera dokładny opis metod działania programu, zastosowanych algorytmów, rozmieszczenia i sposobu działania poszczególnych komponentów. Mogą się w niej znajdować fragmenty kodów źródłowych, graficzne reprezentacje algorytmów, diagramy przepływu danych, opisy UML czy XML.

Dokumentacja techniczna powinna umożliwić ponowne użycia raz napisanego kodu i ułatwić śledzenie i debugowanie tego kodu.

Dokumentacja techniczna powinna zawierać opis:

- wykorzystanych technologii (na przykład XHTML, PHP, CSS),
- wykorzystanych bibliotek (na przykład jQuery),
- zastosowanych wzorców projektowych, architektury aplikacji (na przykład MVC),
- zdefiniowanych użytkowników systemu (na przykład *admin*, *user*),
- zdefiniowanych funkcjonalności dla każdego użytkownika.

Dokumentacja techniczna może zawierać:

- kod źródłowy aplikacji,
- graficzną reprezentację algorytmów.

Dokumentacja użytkownika

Dokumentacja użytkownika to opis aplikacji przeznaczony dla jej użytkownika. Ma ona wyjaśnić, jak powinien działać program i jak z niego korzystać. Na tę dokumentację składają się na przykład pliki pomocy, ogólne informacje o aplikacji i sposobie jej obsługi.

Dokumentacja użytkownika jest tworzona po zakończeniu procesu budowania aplikacji.

Podstawowe elementy dokumentacji użytkownika to:

- *Opis funkcjonalny*. Ta część dokumentacji powinna określać przeznaczenie i główne możliwości aplikacji. Opis funkcjonalny powinien zawierać niezbędne informacje pozwalające ocenić, czy aplikacja zaspokaja konkretne potrzeby użytkownika.
- *Podręcznik użytkownika*. Jest to opis aplikacji przeznaczony głównie dla jej użytkowników. Część ta powinna wyjaśniać:
  - jak rozpoczynać i kończyć pracę z aplikacją,
  - jak obsługiwać najczęściej wykorzystywane funkcje aplikacji,
  - jak obsługiwać błędne operacje wykonane przez użytkownika,
  - jak korzystać z systemu pomocy.

Podręcznik użytkownika powinien zawierać prosty przykład korzystania z aplikacji.

Dokumentacja administratorów systemów

Dokumentacja przeznaczona dla administratorów systemów powinna zawierać:

- *Kompletny opis aplikacji*. W tej części powinny się znajdować:
  - szczegółowy opis wszystkich funkcji aplikacji,
  - opis formatów danych,
  - opis błędów, które mogą się pojawić podczas pracy z systemem,
  - informacje o wszelkich ograniczeniach związanych z wykorzystaniem aplikacji.
- *Opis instalacji*. Część ta powinna zawierać opis procedury instalacji oraz dostrojenia aplikacji do środowiska, w którym będzie pracować.
- *Podręcznik administratora systemu*. Część ta powinna opisywać możliwości zmian konfiguracji systemu i sposoby udostępniania aplikacji użytkownikom.

8.3. Automatyczne generowanie dokumentacji użytkownika

Opracowywanie dokumentacji technicznej jest procesem czasochłonnym, dlatego wiele środowisk programistycznych wspomaga tworzenie dokumentacji i umożliwia automatyczne jej generowanie.

8.3.1. Narzędzia automatycznego generowania dokumentacji

JavaDoc

JavaDoc to narzędzie automatycznego generowania dokumentacji na podstawie zamieszczonych w kodzie źródłowym, napisanym w języku Java, specjalnie sformatowanych komentarzy. Program automatycznie dołącza informacje o nazwach komentowanych klas, metod i zmiennych. Komentarze umieszczone w kodzie programu muszą być zapisane w formacie DocBlocks. Otrzymane dokumenty są zapisane w formacie HTML.

Doxygen

Doxygen to narzędzie do dokumentowania oprogramowania pisanego w językach C++, C, Java, Python, PHP, C#. Dokumentacja jest generowana na podstawie analizy kodu i komentarzy. Program wykorzystuje komentarze DocBlocks. Otrzymana dokumentacja może zostać zapisana w formacie RTF, PostScript, PDF i HTML.

Dokumentacja może być generowana zarówno bezpośrednio ze źródła, jak i z plików połączonych z nim.

phpDocumentor

phpDocumentor to narzędzie służące do tworzenia dokumentacji aplikacji napisanych w języku PHP. Dokumentacja jest generowana na podstawie specjalnie sformatowanych komentarzy (format DocBlocks) wewnątrz kodu źródłowego.

phpDocumentor wspiera zarówno kod napisany obiektowo, jak i strukturalnie. Tworzy dokumentację w formacie HTML i PDF.

8.3.2. ESDoc

ESDoc to narzędzie do generowania dokumentacji aplikacji napisanych w języku JavaScript. Jeżeli do programu nie zostały dodane komentarze, to dokumentacja zostanie wygenerowana tylko na podstawie kodu. Jeśli w kodzie zostaną umieszczone komentarze, które opiszą poszczególne jego fragmenty, to dokumentacja będzie wygenerowana na podstawie kodu i komentarzy. Program wykorzystuje komentarze DocBlocks.

Poniższy przykład pokazuje sposób dołączania komentarzy do opisu funkcji.

Przykład 8.5

Do definicji funkcji zapisanej w języku JavaScript został dodany komentarz opisujący, czego dotyczy funkcja i jakie przyjmuje argumenty.

```
/**
 * To jest klasa Licz.
 */
export default class Licz {
 /**
  * @param {number} a - liczba a.
  * @param {number} b - liczba b.
  * @return {number} wynik sumowania a i b.
  */
  sum(a, b) {
    return a + b;
  }
}
```

Instalacja ESDoc

Do zainstalowania aplikacji ESDoc wymagane jest zainstalowanie środowiska NPM.

Jeżeli środowisko NPM jest dostępne, w wierszu poleceń należy wpisać:

```
npm install -g esdoc
```

co spowoduje zainstalowanie ESDoc.

Następne polecenie:

```
cd projekt/
```

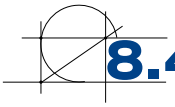
spowoduje przejście do katalogu z projektem.

Kolejny krok to utworzenie pliku konfiguracyjnego `esdoc.json`:

```
echo {"source": "./src", "destination": "./doc"} > .esdoc.json
```

Do wygenerowania dokumentacji za pomocą ESDoc należy wpisać polecenie:

```
esdoc
```



8.4. Pytania i zadania

8.4.1. Pytania

1. Jakie elementy powinien zawierać komentarz w formacie DocBlocks?
2. Jak wygląda zapis znacznika w komentarzu DocBlocks?
3. Jakie dokumenty powinny wchodzić w skład dokumentacji aplikacji?
4. Co powinna zawierać dokumentacja techniczna aplikacji?
5. Jakie elementy powinny znaleźć się w dokumentacji użytkownika aplikacji?
6. Wymień poznane narzędzia do automatycznego generowania dokumentacji.
7. Jakiego rodzaju dokumentację można tworzyć za pomocą narzędzi do automatycznego generowania dokumentacji?

Bibliografia

[1] Keith Wald, Jason Lengstorf, *PHP i jQuery. Techniki zaawansowane*, Helion, Gliwice 2017.

[2] Jolanta Pokorska, *Kwalifikacja E.14. Tworzenie aplikacji internetowych. Podręcznik do nauki zawodu technik informatyk*, Helion, Gliwice 2014.

[3] Jolanta Pokorska, *Kwalifikacja EE.09. Programowanie, tworzenie i administrowanie stronami internetowymi i bazami danych. Część 4. Tworzenie aplikacji internetowych. Podręcznik do nauki zawodu technik informatyk*, Helion, Gliwice 2019.

Źródła internetowe

[1] *https://www.w3schools.com/js/*

[2] *https://www.w3schools.com/jsref/default.asp*

[3] *https://angular.io/start*

[4] *https://www.javatpoint.com/angular-8*

[5] *https://angular.io/cli*

[6] *https://www.w3schools.com/nodejs/*

[7] *https://www.w3schools.com/react/*

[8] *https://www.w3schools.com/jquery/*

[9] *https://www.php.net/manual/en/index.php*